# Controller-Guided ChEMBL Search for Agentic Molecular Design

Experiment 9 · 13 October 2026

**Abstract**

We investigate whether coordinated database search helps molecular-design agents discover useful compounds and learn reusable strategies. Our controller-guided architecture couples agentic ChEMBL search with persistent task and tool memory. The controller and database agent refine a search brief through a bounded dialogue channel; the database agent then writes SQL and SMARTS queries over a 50,000-compound ChEMBL slice containing structures, bioactivity measurements, assay context, and publication titles. Curator roles select candidate molecules, extract conditional transformation strategies, and retain query lessons. The controller uses this memory to choose further consultations or independently executed worker edits. Source labels support analysis of the resulting search. In a 40-cycle case study with a local amlodipine multi-property oracle, 17 database consultations and 69 edit attempts raise the best score from 0.4910 to 0.8605. Retrieval supplies nicardipine and subsequently the final best compound, amlodipine besylate; the best generated product scores 0.8110. Database-derived strategies improve GuacaMol-lineage parents in 5/14 attempts, compared with 3/16 for controller/feedback strategies. Final-best discovery requires 87 scored-candidate evaluations, or 131 literal oracle calls including validation rechecks, excluding the 100 initialization evaluations. These results support coordinated database search as a practical source of both candidates and actionable design knowledge, with controlled multi-task studies needed to establish reproducible gains.

## 1 Introduction

Molecular-design agents must decide which structures to explore, which transformations to test, and when to abandon an unproductive direction. Scientific language models can express these decisions as hypotheses and revise them from objective feedback. PABLO demonstrates how a hierarchy of reasoning and generation agents can implement this optimization loop [1]. We investigate a complementary systems question: *does adding a dedicated, controller-coordinated database agent improve molecular search by acquiring useful compounds and reusable design strategies?*

ChEMBL offers a structured source of chemical evidence: compounds are linked to measured activities, assay descriptions, targets, and publications [2]. Access alone does not determine how this evidence should enter an optimization workflow. A useful database agent must translate a changing scientific need into executable queries, interpret returned rows, recover from failed searches, and deliver information the design controller can act on. Coordination matters because a request for a new scaffold differs from a request for evidence about a particular substitution pattern.

Our architecture addresses this interface through a dialogue channel, an iterative SQL/SMARTS search loop, and persistent agent memory. The database agent supplies candidate molecules for adoption and conditional strategies for later edits. Separate curators turn retrieved chemistry and tool-use experience into compact, reusable knowledge; the controller decides when to search again and when to test a strategy. Experiment 9 demonstrates both routes to improved search under a limited oracle budget. The study compares its outcomes with earlier local agent-only runs while treating those runs as historical context, since their prompts and policies differ. The contribution is the coordinated search-and-memory architecture and its measured case-study results.

## 2 Architecture and Methodology

### 2.1 Controller-led search and candidate execution

The controller receives the task, scored candidate pool, persistent memory, and decision history. Each cycle it chooses either a database consultation, expressing a scientific need, or an edit cycle, selecting one parent and up to three transformation instructions. The database agent translates the need into a searchable brief and executes retrieval; worker calls produce the requested structures. Oracle feedback informs the next controller decision.

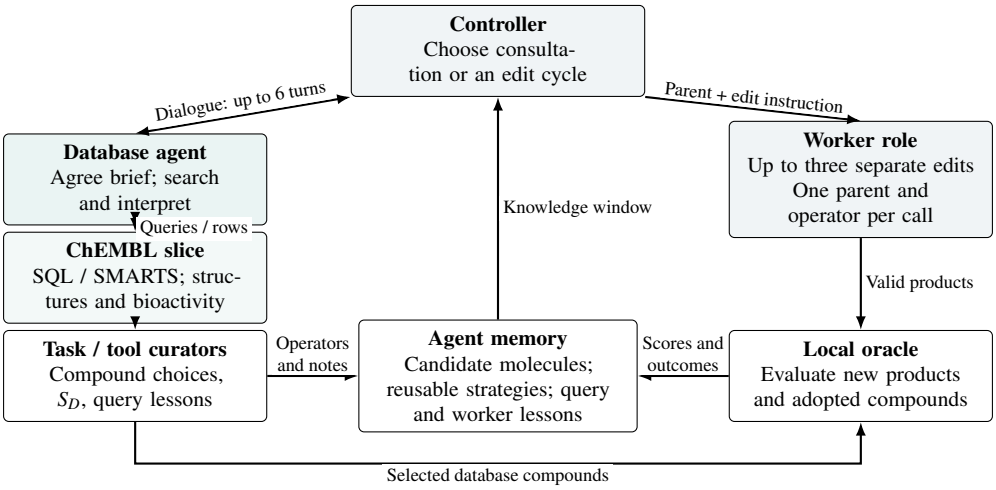



**Figure 1:** Controller-guided database search and candidate execution. A bounded dialogue establishes the search brief; the database agent queries ChEMBL, and curators convert the evidence into molecules, strategies, and query lessons. The controller uses persistent memory to schedule the next consultation or worker edit.

Persistent task memory stores candidate molecules and reusable operators specifying a transformation, applicability condition, exploration category, and supporting evidence. Task curators read retrieved compounds in batches of ten and propose molecules and strategies; tool curators retain query errors and execution lessons. Adoption choices precede new oracle scores. Full memory remains on disk, while prompt windows expose useful candidates and strategies without discarding archived labels. Source labels ($G/D$ for starting molecules and $S_I/S_D$ for controller/feedback versus database-derived strategies) support descriptive analysis.

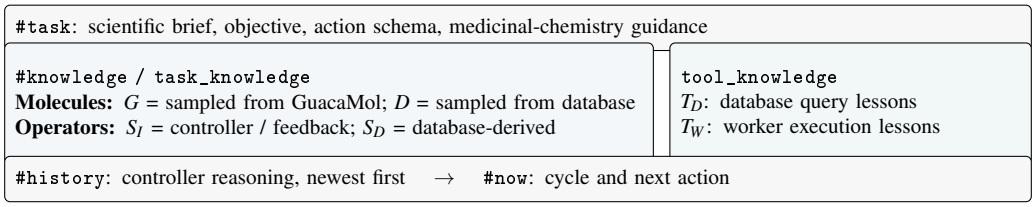



**Figure 2:** Persistent memory in the controller prompt. Task knowledge exposes scored candidates and applicable strategies; tool knowledge retains lessons for future searches and worker calls. Reasoning history and the current action follow the knowledge window.

### 2.2 ChEMBL dataset and query interface

The local ChEMBL 37 slice contains 50,000 valid structures with free-base molecular weight 150–650, AlogP $-1$ to 6, and at least one $IC_{50}$, $K_i$, $K_d$, $EC_{50}$, or $GI_{50}$ measurement. Selection prioritizes clinical phase and retains up to 15 measurements and three publication titles per compound. Four linked tables expose molecules, activities, assays, and documents. SQL supports metadata and bioactivity filtering; SMARTS supports structural queries across the slice.

### 2.3 Dialogue, iterative search, and feedback

A consultation starts with the controller's abstract need. The database agent restates the brief, asks clarifying questions, and rates its understanding; the controller responds and rates satisfaction. Dialogue ends when both reach 8/10 or after six turns. This channel separates agreement about what to find from execution of the search.

On cycle 21 the controller asked for dihydropyridines that differ from amlodipine in the 4-substituent or the ester. The database agent restated that chemistry and rated its understanding 9/10, but its plan was still a comparison of three named drugs. The controller rated satisfaction 5/10 and answered: "your plan to retrieve only Amlodipine, Nifedipine, and Nicardipine is too narrow ... search for all molecules in this class." The next turn dropped the three-name list. The agent proposed a search by the *dipine* suffix or by structural class, and the controller set satisfaction to 9/10: attempt the broader query, and report a three-drug result only if that query finds nothing else. The agreed brief was no longer an abstract comparison of famous drugs. It was a retrieval of every dihydropyridine in the slice.

The database agent then follows a ReAct-style loop [4] for up to six rounds, each allowing five read-only SQL statements and three SMARTS patterns, capped at 20 rows per query. An LLM relevance rating retains compounds at 7/10 or above; it is

distinct from the molecular oracle. Search observations and errors inform subsequent queries and tool-memory updates.

Cycle 13 shows that revision inside one consultation. The brief asked for canonical 1,4-dihydro-2,6-dimethylpyridine-3,5-dicarboxylates with a 4-aryl substituent. Round 1 issued three SQL statements, all interrupted, and one SMARTS pattern for that fully substituted core. The pattern returned 0 rows, and the holding set stayed empty. The agent recorded that the pattern was too specific and replaced it, in round 2, with the bare 1,4-dihydropyridine ring `[NX3]1[CX3]=[CX3][CX4][CX3]=[CX3]1`. That query returned 20 rows, and 17 compounds were kept. The same loop, after the cycle-21 brief above, is what later widened a three-name lookup into the 18-compound set that contained amlodipine besylate.

Workers receive a parent SMILES and one instruction, without its score or the strategy evidence. Up to three edits are tested separately, and their outcomes return to the controller before the next cycle.

## 3 Experimental Protocol and Oracle Budget

### 3.1 Task, models, and evaluation conventions

Experiment 9 uses Qwen3.8-27B for 40 controller cycles, with temperatures 0.4 for reasoning and 0.8 for workers, and thinking disabled. The first 100 GuacaMol v1 training molecules are scored; their top ten initialize the candidate pool (best 0.4910). The prompt names the amlodipine family and drug class, while withholding the objective formula.

The local amlodipine multi-property oracle follows GuacaMol's similarity and three-ring objective [3], using count-based Morgan fingerprints of radius two:

$$f(x) = \left[\text{Tanimoto}(\phi(x), \phi(x_{\text{aml}}))\exp\left(-\frac{1}{2}\left(\frac{r(x)-3}{0.5}\right)^2\right)\right]^{1/2}, \tag{1}$$

Here $r(x)$ is the RDKit ring count. We report individual scores from this implementation, not the official top-1/10/100 aggregate. Parent improvement means $f(x_{\text{product}}) > f(x_{\text{parent}})$, not necessarily a new global best.

### 3.2 Oracle calls relative to the published PABLO protocol

PABLO reports a fixed 20,000-evaluation budget, not average calls to reach a target score [1]. Table 1 subtracts its nominal 100-molecule initialization and compares that budget with experiment 9. Differences in models, task information, oracle implementation, and stopping criteria preclude a matched efficiency claim.

Experiment 9 makes 239 post-initialization calls: 61 edit-product scores, 75 adoption checks, and 103 validation rechecks. Final-best discovery takes 131 calls. Candidate-level counts are 136 and 87, respectively; both conventions exclude the 100 initialization evaluations.

**Table 1:** PABLO's published budget versus measured calls in **our architecture**. Initialization is shown separately. PABLO is a fixed budget, not an average stopping time; our calls include validation rechecks.

| Method / endpoint | Calls after initialization | Calls including initialization |
|---|---|---|
| PABLO published benchmark budget [1] | 19,900 | 20,000 |
| **Our architecture: final-best discovery** | **131** | **231** |
| **Our architecture: complete experiment 9** | **239** | **339** |

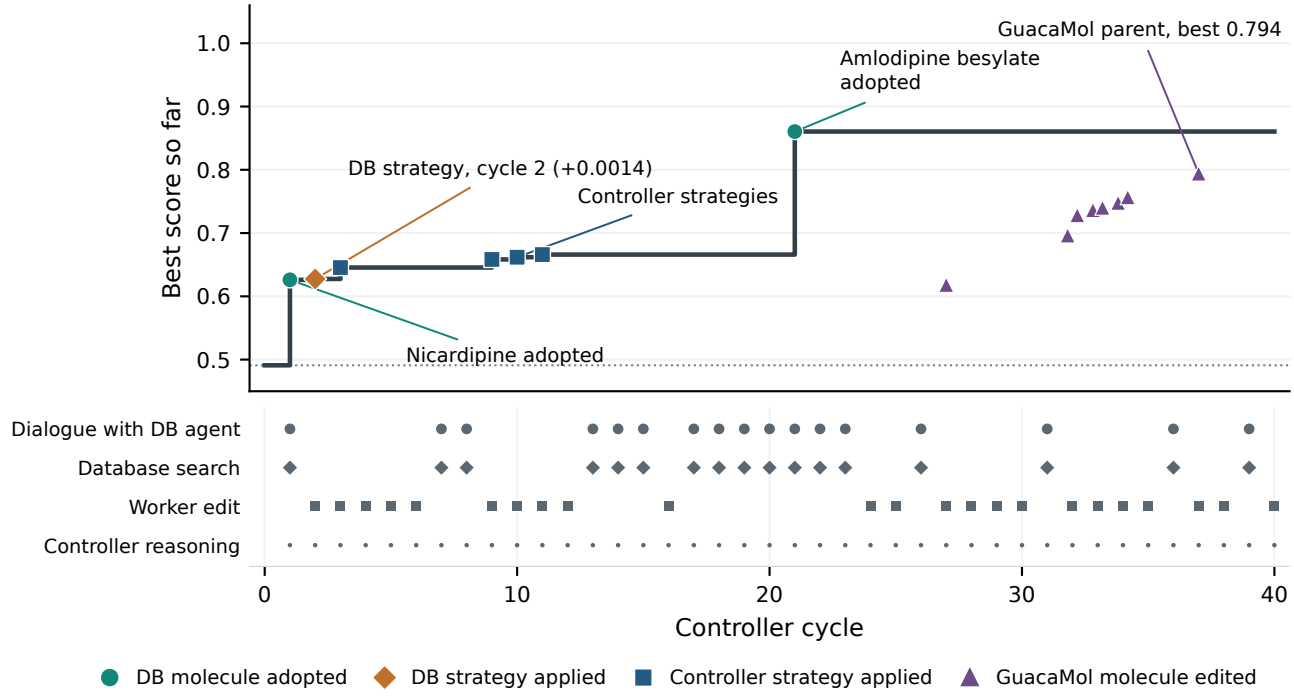



**Figure 3:** Forty controller cycles as one trajectory. The step is the best score in the pool. Filled markers are the events that raised it: a database molecule adopted (cycles 1 and 21), a database strategy applied (cycle 2, +0.0014, drawn at the same height as the cycle-1 adoption), and controller strategies (cycles 3 and 9–11). Triangles are edits that improved a GuacaMol-lineage parent, plotted at the product score; the best is 0.7941 at cycle 37 and does not replace amlodipine besylate. The strip marks, once per cycle, controller reasoning, dialogue with the database agent, database search, and worker execution.

## 4 Results: Database-Assisted Molecular Search

### 4.1 Candidate acquisition and optimization progress

Across 17 consultations, the database agent issues 95 SQL statements and 43 SMARTS patterns. Search offers 172 compound records and leads to 75 adoption checks, including revisits; 16 distinct adopted structures enter the pool. Nicardipine (CHEMBL1484), retrieved at cycle 1, raises the best score to 0.6261. Edits then reach 0.6660 by cycle 11. At cycle 21, retrieval supplies amlodipine besylate (CHEMBL1200402) at 0.8605, which remains best through cycle 40.

The historical local agent-only best is 0.5774 (Table 2). Experiment 9 gains 0.2831 over that result and 0.3695 over initialization, while its best generated product scores 0.8110. These historical comparisons motivate the architecture, but do not isolate a causal database-agent effect.

**Table 2:** Historical local comparison. Agent-only rows are local PABLO-style variants, not published PABLO results. Prompts and budgets differ; evaluations exclude initialization. **Our architecture** is bold.

| Architecture / recorded run | Cycles | Evaluations | Best score | Gain from start |
|---|---|---|---|---|
| Agent-only, five-candidate batches (exp. 1) | 5 | 15 | 0.5368 | +0.0458 |
| Agent-only, reason–worker–observe (exp. 3) | 8 | 8 | 0.5334 | +0.0424 |
| Agent-only, adaptive history (exp. 5) | 85 | 58 | 0.5774 | +0.0864 |
| **Controller + database agent (exp. 9)** | **40** | **136** | **0.8605** | **+0.3695** |

### 4.2 Learning and applying database-derived strategies

Curators turn retrieved findings into reusable transformations with applicability conditions. The controller tests them on selected parents; Table 3 and Figure 4 summarize the outcomes.

**Table 3:** Reuse of database-derived strategies and retrieved starting molecules. Rates include all attempts; deltas omit invalid and duplicate outcomes. Best denotes an edited product. Bold rows use **database-derived strategies or compounds**.

| Starting molecule | Strategy source | Attempts | Scored | Improved | $\sum \Delta f$ | Best product |
|---|---|---|---|---|---|---|
| Sampled from GuacaMol | Controller/feedback | 16 | 15 | 3 (18.8%) | −1.9410 | 0.7479 |
| **Sampled from GuacaMol** | **Database-derived** | **14** | **11** | **5 (35.7%)** | −1.5501 | **0.7941** |
| **Sampled from database** | **Controller/feedback** | **21** | **19** | **4 (19.0%)** | −1.4307 | **0.7941** |
| **Sampled from database** | **Database-derived** | **18** | **16** | **1 (5.6%)** | −1.2773 | **0.8110** |

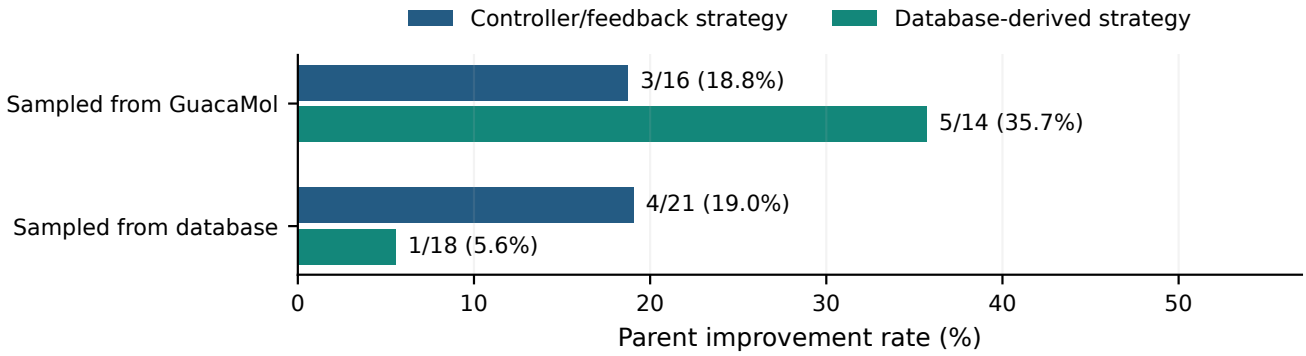



**Figure 4:** Outcomes of reusable design strategies. Database-derived strategies improve GuacaMol-lineage parents more often in this run. All attempts enter the denominator; adaptive selection prevents a causal interpretation.

Database-derived strategies improve 5/14 GuacaMol-lineage parents, compared with 3/16 controller/feedback attempts; their best product scores 0.7941. The strongest generated product (0.8110) also uses a database-derived strategy. On retrieved lineages, improvement rates reverse (1/18 versus 4/21). All groups have negative summed deltas: useful strategy applications occur, but average edit quality does not uniformly improve.

## 5 Discussion and Next Study

### 5.1 What coordinated database search contributes

The architecture turns structured chemical evidence into operational search state. A consultation can change the available candidate pool, add a transformation the controller can test, and retain lessons about how to obtain useful rows next time. These outputs persist beyond a single retrieval session. The controller can therefore alternate evidence acquisition and candidate execution as the search develops, instead of treating database access as a fixed preprocessing step.

Experiment 9 illustrates the complementarity of these outputs. Retrieved compounds establish better starting states and provide the final best candidate. An operator learned from the database, SD2, proposes varying small alkyl ester substituents; it is applied 21 times and improves five parents, with a largest individual gain of approximately 0.110. Four improvements occur in eleven applications to GuacaMol lineages. This supports the practical value of storing retrieved findings as reusable instructions, while the failures show that a plausible chemical strategy does not guarantee an improved objective score.

### 5.2 Limits of the present evidence

**Matched controls and replication are still needed.** This is one run on one task with one model. Historical agent-only runs differ in task information, memory, search policy, and budget. Adaptive selection of parents and strategies makes within-run comparisons descriptive. A common controller also carries knowledge between search and editing decisions, so source labels alone do not isolate causal contributions.

**The strongest result is known-compound retrieval.** The task names the drug family and the slice prioritizes clinically advanced compounds. Retrieving amlodipine besylate demonstrates effective database-assisted search, not de novo drug discovery. The oracle includes its disconnected counterion, affecting fingerprints and ring count. Scores do not establish biological potency or clinical utility.

**Learned strategies need chemical validation.** Retrieved assays differ in context, and LLM summaries can misinterpret structures or activity evidence. A recorded SD15 statement misidentifies substituent positions. Expert review and structure-

aware checks are necessary before treating such instructions as validated medicinal-chemistry rules.

### 5.3 Testing generality across models and tasks

The architecture should be tested on other GuacaMol tasks and with several closed-source models. Within each model and task, hold the prompt, initial molecules, memory policy, and raw oracle budget fixed across four conditions: agent-only, database strategies without compound adoption, compound adoption without database strategies, and the full system. Multiple seeds and database slices excluding reference compounds and close analogues would test whether learned strategies remain useful when retrieval cannot return a strong known candidate. Compare best-score trajectories and area under a common budget, together with validity, diversity, and chemical feasibility. Randomized strategy tests on shared parents, isolated control contexts, and fragment-standardized scoring would improve interpretability.

## 6 Conclusion

A controller-coordinated ChEMBL agent provides an actionable interface between structured chemical evidence and molecular optimization. Through dialogue, SQL/SMARTS search, curation, and persistent memory, it supplies both promising compounds and strategies for subsequent edits. In experiment 9, the best score rises from 0.4910 to 0.8605, and generated products reach 0.8110. Final-best discovery takes 87 candidate evaluations or 131 literal oracle calls, excluding 100 initialization evaluations. The case study supports agentic database search as a useful component of molecular-design systems; replicated, budget-matched evaluations across tasks and models are needed to establish the scope of its benefits.

## References

[1] N. Maus et al. *Purely Agent-Driven Black-Box Optimization for Biological Design*. arXiv:2601.22382, 2026. arxiv.org/abs/2601.22382.

[2] B. Zdrazil et al. The ChEMBL Database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 2024. doi:10.1093/nar/gkad1004.

[3] N. Brown, M. Fiscato, M. H. S. Segler, and A. C. Vaucher. GuacaMol: Benchmarking models for de novo molecular design. *Journal of Chemical Information and Modeling*, 59(3):1096–1108, 2019. doi:10.1021/acs.jcim.8b00839.

[4] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. arxiv.org/abs/2210.03629.